Anonymous & Callback Functions

Anonymous Function

A function **without a name** *that is stored in a variable.*

```
const greet = function() {
  console.log("Hello");
};
```

How to write and store an anonymous function.

Callback Function

A function passed into another function as an argument, to be executed (called back) later.

Timers: setTimeout & setInterval

setTimeout()

Runs a function once in the future. Takes 2 params: the **function**, *and time in* **milliseconds**.

```
setTimeout(function() {
  console.log('timeout');
}, 3000);
```

Waits exactly 3 seconds (3000ms), then runs once.

setInterval()

Exactly like setTimeout, but runs the function **again and again** *periodically.*

clearInterval()

Kills a setInterval loop. Requires the ID/reference to stop it.

Synchronous vs Asynchronous Code

Synchronous

Waits for one line to finish completely before going to the next line.

Asynchronous

Won't wait *for a line to finish before going to the next. (e.g., setTimeout runs in the background).*

Arrow Functions

A modern, shorter way to write functions. ⚠ Arrow functions do not support hoisting.

```
const arrow = () => {
  console.log('hello');
};
```

Basic arrow function syntax.

Shortcut 1: One Parameter

If exactly one parameter, you can drop the parenthesis.

```
const double = number => {
  return number * 2;
};
```

Notice **number** *has no parenthesis around it.*

Shortcut 2: Implicit Return

If the function only has a return statement, you can drop the curly braces and the `return` keyword.

```
const add = (a, b) => a + b;
```

Returns **a + b** *automatically.*

Array Iteration Methods

forEach(): *Loops through an array. ⚠ You can't stop this loop with break or continue.*

```
array.forEach((value, index) => {
  // code
});
```

value *(Required): The current item.* **index** *(Optional): Position starting at 0.*

filter(): *Creates a brand-new array containing only items that pass a specific test. Original array is unchanged.*

```
[1, -3, 5].filter(val => val >= 0);
```

→ **[1, 5]** *(Keeps only positive numbers).*

map(): *Transforms an array into another array. Whatever you return is added to the brand-new array.*

```
[1, 1, 3].map(val => val + 10);
```

→ **[11, 11, 13]** *(Adds 10 to every item).*

Closures

What is it?

If a function has access to a value, it will **always** *have access to that value. The value gets packaged (enclosed) with the function.*

How it works:

A closure gives an inner function access to the outer function's scope, **even after the outer function has finished executing**.